

AWiR – ćwiczenie 4

Temat:

Obsługa żądań REST (REST service, JSON, walidacja, CRUD, OpenAPI)

Cel ćwiczenia:

Rozszerzenie aplikacji z ćwiczenia 3 o **warstwę REST** (równoległe do istniejącego MVC/Thymeleaf):

- publikacja CRUD dla encji User pod /api/users,
- walidacja danych wejściowych z odpowiedziami JSON,
- dokumentacja i interaktywne testy w **Swagger UI** (springdoc-openapi),
- podstawowe testy kontrolera REST.

Uwaga: nie usuwamy istniejących widoków i kontrolera MVC – REST działa obok, pod /api/....

Niezbędne narzędzia:

1. DE: STS 4 / IntelliJ / VS Code
2. Maven 3.9+
3. JDK 21
4. Spring Boot 3.3.x
5. Baza: H2 (z ćw. 3)

Projekt z ćwiczenia 3 zawierał:

- @Entity **User** (id: Long, name, email) z walidacją,
- UserRepository extends JpaRepository<User, Long>,
- UserService z metodami: save, findAll, findById, delete,
- kontroler MVC UserController + widoki,
- konfigurację H2 w application.properties.

W tym ćwiczeniu dodaj:

- nowy **kontroler REST** (UserRestController) w pakiecie web.rest z pełnym CRUD,
- **globalny handler błędów** (RestExceptionHandler) zwracający czytelne komunikaty JSON,
- zależność do **springdoc-openapi** i uruchomienie **Swagger UI** na /swagger-ui/index.html,
- przykładowe testy REST (MockMvc lub Postman).

Nowa struktura projektu

```
src/main/java/edu/zut/awir/
├── AwirApplication.java # klasa główna (z ćw. 3)
├── controller/
│   └── UserController.java # kontroler MVC (z ćw. 2/3)
├── model/
│   └── User.java # encja JPA (z ćw. 3; wcześniej zwykła klasa z ćw. 2)
├── repository/
│   └── UserRepository.java # JPA repo (z ćw. 3)
├── service/
│   └── UserService.java # warstwa serwisu (z ćw. 3)
└── web/rest/
    ├── UserRestController.java # NOWE: REST CRUD
    └── RestExceptionHandler.java # NOWE: globalny handler błędów JSON

src/main/resources/
├── application.properties # konfiguracja H2/JPA (z ćw. 3)
└── templates/ # widoki MVC (pozostają z poprzednich ćwiczeń)
    ├── add-user.html # z ćw. 2 (formularz dodawania)
    ├── user-info.html # z ćw. 2 (widok informacji)
    ├── user-list.html # z ćw. 3 (lista)
    └── edit-user.html # z ćw. 3 (edycja)
```

1. Rozszerzenie zależności w pliku pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <!-- dokumentacja REST /swagger-ui.html -->
  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.6.0</version>
  </dependency>
</dependencies>
```

2. Kontroler REST

src/main/java/edu/zut/awir/web/rest/UserRestController.java

```
package edu.zut.awir.web.rest;

import edu.zut.awir.model.User;
import edu.zut.awir.service.UserService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
```

```

@RequestMapping("/api/users")
@RequiredArgsConstructor
public class UserRestController {

    private final UserService service;

    @GetMapping
    public List<User> list() {
        return service.findAll();
    }

    @GetMapping("/{id}")
    public User get(@PathVariable Long id) {
        var u = service.findById(id);
        if (u == null) throw new NotFoundException("User",
            id);
        return u;
    }

    @PostMapping
    public ResponseEntity<User> create(@Valid @RequestBody
    User payload) {
        payload.setId(null); // ID generowane przez bazę
        var saved = service.save(payload);
        return
    ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @PutMapping("/{id}")
    public User update(@PathVariable Long id, @Valid
    @RequestBody User payload) {
        var existing = service.findById(id);
        if (existing == null) throw new
    NotFoundException("User", id);
        payload.setId(id);
        return service.save(payload);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        var existing = service.findById(id);
        if (existing == null) throw new
    NotFoundException("User", id);
        service.delete(id);
    }

    public static class NotFoundException extends
    RuntimeException {
        public NotFoundException(String what, Object id) {
            super(what + " " + id + " not found");
        }
    }
}

```

```
}
```

3. Globalny handler błędów

src/main/java/edu/zut/awir/web/rest/RestExceptionHandler.java

```
package edu.zut.awir.web.rest;

import
edu.zut.awir.web.rest.UserRestController.NotFoundException;
import jakarta.validation.ConstraintViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import
org.springframework.web.bind.MethodArgumentNotValidException;
import
org.springframework.web.bind.annotation.ExceptionHandler;
import
org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

@RestControllerAdvice
public class RestExceptionHandler {

    public record ApiError(Instant timestamp, int status, String
error,
String message, String path, Map<String,String> fieldErrors)
{}

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<ApiError>
handleNotFound(NotFoundException ex, WebRequest req) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), req,
null);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError>
handleInvalid(MethodArgumentNotValidException ex, WebRequest
req) {
        Map<String,String> fields = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(fe ->
fields.put(fe.getField(), fe.getDefaultMessage()));
        return build(HttpStatus.BAD_REQUEST, "Validation failed",
req, fields);
    }

    @ExceptionHandler(ConstraintViolationException.class)
```

```

public ResponseEntity<ApiError>
handleInvalid2(ConstraintViolationException ex, WebRequest
req) {
    Map<String,String> fields = new HashMap<>();
    ex.getConstraintViolations().forEach(cv ->
    fields.put(cv.getPropertyPath().toString(), cv.getMessage()));
    return build(HttpStatus.BAD_REQUEST, "Validation failed",
    req, fields);
}

private ResponseEntity<ApiError> build(HttpStatus status,
String message, WebRequest req,
Map<String,String> fieldErrors) {
    var path =
    Optional.ofNullable(req.getDescription(false)).orElse("")
    ;
    var body = new ApiError(Instant.now(), status.value(),
    status.getReasonPhrase(),
    message, path, fieldErrors);
    return ResponseEntity.status(status).body(body);
}
}

```

4. Testowanie REST

1) Lista wszystkich użytkowników

```
curl http://localhost:8080/api/users
```

2) Dodanie

```
curl -X POST http://localhost:8080/api/users \
-H "Content-Type: application/json" \
-d '{"name":"Alice","email":"alice@example.com"}'
```

3) Pobranie po ID

```
curl http://localhost:8080/api/users/1
```

4) Aktualizacja

```
curl -X PUT http://localhost:8080/api/users/1 \
-H "Content-Type: application/json" \
-d '{"name":"Alice Updated","email":"alice@ex.com"}'
```

5) Usunięcie

```
curl -X DELETE http://localhost:8080/api/users/1 -i
```

6) Błędy walidacji (przykład)

```
curl -X POST http://localhost:8080/api/users \
-H "Content-Type: application/json" \
```

```
-d '{"name":"","email":"not-an-email"}'
```

Testy SwaggerUI

<http://localhost:8080/swagger-ui/index.html>

Testy kontrolera REST

src/test/java/edu/zut/awir/web/rest/UserRestControllerTest.java

```
package edu.zut.awir.web.rest;
```

```
import com.fasterxml.jackson.databind.ObjectMapper;
import edu.zut.awir.model.User;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest
;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilder
s.post;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.
status;
```

```
@WebMvcTest(UserRestController.class)
class UserRestControllerTest {
    @Autowired MockMvc mvc;
    @Autowired ObjectMapper om;

    @Test
    void rejectsInvalidPayload() throws Exception {
        var u = new User(null, "", "not-an-email");
        mvc.perform(post("/api/users")
            .contentType(MediaType.APPLICATION_JSON)
            .content(om.writeValueAsString(u)))
            .andExpect(status().isBadRequest());
    }
}
```

5. Zadania do wykonania

- W kontrolerze REST użyj DTO (np. UserDto) zamiast encji User; dodaj mapowanie UserDto->User.
- Dodaj **pole unikalne** (np. email UNIQUE) i obsłuż konflikt (409 CONFLICT).